

CUDA PROGRAMMING FOR NVIDIA H100S

Kernel 启动

Prateek Shukla · 简体中文版

一些重要的约束与函数

`__cluster_dims__(x,y,z)` 用于编译期线程块集群形状。

`__launch_bounds__(maxThreads[,minBlocksPerSM[,maxBlocksPerCluster]])`。第 3 个参数是集群相关的那个，在 SM90 上更重要。

`__maxnreg__(N)` 用于限制每线程寄存器数量。

`__grid_constant__` 用于只读的 grid 生命周期 kernel 参数。这在 CUDAMap / TMA 描述符场景中尤其常见。

`__forceinline__` 强制 nvcc 在单个翻译单元内内联该函数。

`__restrict__` 告诉 nvcc：在该指针于当前作用域中的生命周期内，所指向的内存不会被任何其他用于访问同一数据的指针起别名。

为什么需要多 **kernel** 设置

epilogue 有一个硬上限。强行突破它会摧毁你的性能。融合操作会与 mainloop 的累加器 tile 竞争，可能造成寄存器压力。需要跨 tile 依赖的操作（LayerNorm、Softmax）在架构上与 GEMM epilogue 的独立 tile 处理不兼容。数学上严格要求一个 kernel 边界才能正确归约。你必须接受 kernel 边界和中间 DRAM 写入。工程挑战不是强行做成单体 kernel，而是让 kernel 之间的交接几乎零成本。这要靠部署依赖启动协议、L2 预取策略和跨 grid mbarrier 来实现

流串行化问题

通常，一个 grid 必须完全退出之后，同一流上的下一个 grid 才能开始发射工作。这种严格的完成排序造成了利用率缺口：当第一个 grid 进入其低占用率尾部时尤为明显。硬件调度器把同流的 grid 当作单个有序队列。在 Kernel A 报告全局完成之前，它不会向 Kernel B 分发线程块（thread block）槽位，即使一些 SM 已经利用不足。随着 Kernel A 接近完成，只剩不断萎缩的 SM 子集仍有活动工作。其余 SM 闲置，因为在默认流规则下没有任何下一 grid 的线程块具备资格。交接是一个整 grid 退出事件，而不是逐 tile 的传递。这意味着所有依赖安全的重叠机会都被忽略，除非显式启用依赖启动控制。关键代价是尾延迟放大：在 Kernel A 的最后阶段，硅片还在，却没有发射有用的指令。

硬件信号（**GDC** 指令）

GDC 指令在 GPU 执行时间线内控制 grid 交接的时机。其物理目的是把"调度器可以启动依赖方"与"依赖方可以读取依赖敏感内存"区分开。griddepcontrol.launch_dependents 是由运行中的 warp 发出的、面向调度器的信号。它告诉分发逻辑：依赖 grid 现在已有资格启动，甚至早在活动 grid 全局退出之前。

griddepcontrol.wait 是依赖 grid 中的一道硬执行栅栏。到达该指令的 warp 会被扣住，直到启动依赖条件得到满足，从而防止对尚未定型数据的过早读取。两者共同构成一个两段式协议：提前启动许可加内存安全闸门。调度器可以重叠启动工作，同时栅栏为依赖敏感的加载保持正确性。这些是指令流中的物理控制点，因此摆放位置直接改变重叠时机与硬件竞争。

主机启动权限

只有 CPU 启动包才能为依赖重叠授权放宽的流行为。其物理目的是：除非主机软件显式选择加入，否则保持默认流语义完好。启动时，CPU 把属性写入由 GPU 命令处理器消费的命令描述符。其中一个属性授予在前驱完全退出之前调度依赖 grid 的许可。如果该许可位存在，调度器状态机会把 `launch_dependents` 和 `wait` 当作依赖控制指令来遵循。如果它不存在，调度器执行正常串行化，这些指令没有任何启用效果。因此，交接权限由主机发起、由硬件执行。除非启动元数据授权，设备侧信号无法覆盖流策略。这在混合负载下保护了正确性：某些流需要严格排序，另一些流需要受控重叠。

依赖方的墙（**wait**）

这一阶段定义依赖 grid 中生产者 warp 的早期启动停滞点。其物理目的是让依赖 kernel 提前预留执行上下文，同时阻断不安全的内存流量。

依赖 grid 可以被准入，并在可用的 SM 上开始执行准备指令。生产者 warp 一直运行到抵达 `griddepcontrol.wait`，在那里被硬件停靠。

停靠期间，这些 warp 不能发出依赖敏感的全局读取，例如由活动 kernel 产生的激活张量。这防止了因读取尚未定型数据而导致的缓存填充与内存序违规。

一旦依赖信号得到满足，栅栏释放，同一批 warp 恢复发出加载。结果就是立即向前推进，而不必在生产者完成之后再付出完整的冷启动延迟。

代价是资源被预留：停靠的 warp 仍占用调度槽位，并可能减少其他工作的瞬时余量。

生产者的绿灯（**launch**）

这一阶段是活动 kernel 发出那个精确信号、打开依赖调度的地方。其物理目的是在最后一个正确性安全的时刻触发重叠，同时仍能暴露启动延迟隐藏（latency hiding）的机会。活动 grid 中的生产者 warp 在其剩余写入已推进到足以支持依赖启动时，执行 `griddepcontrol.launch_dependents`。该信号针对的是调度器资格，而不是立即的内存读取许可。在常驻（persistent）kernel 中，正确的摆放位置与最后一个被调度的相关工作 tile 的生命周期绑定。信号应当与真正的流水线完成对齐，而不仅仅是代码区域的词法末尾。信号发出后，依赖线程块可以在活动 grid 排空剩余工作的同时被分发。依赖 grid 的 wait 栅栏仍会守卫依赖敏感的读取，直到所需条件得到满足。信号发得太晚会浪费重叠；发得太早会增加并发压力，并可能降低净吞吐量。

L2 预取技巧（拆分 DMA）

这一阶段通过在依赖 kernel 的 warp 之间划分角色来实现重叠。其物理目的是：当受依赖约束的生产者仍被栅栏拦住时，让内存 fabric 忙于与依赖无关的传输。

一个生产者 warp 到达依赖屏障（barrier）并在激活读取之前暂停。另一个预取 warp 继续运行，为与生产者完成无关的静态权重发出 DMA 式请求。

这些请求的目标是在计算需求之前填充 L2，从而在依赖栅栏打开时减少日后的缺失惩罚。因此，内存系统可以在原本停滞的依赖等待期间预热有用的缓存行。

这种交接是非对称但安全的：激活流量在栅栏后面等待，权重预取则不必。当栅栏释放时，生产者会看到更优的有效延迟，因为部分工作集已经驻留。

只有当预取的数据具有高复用度、并能在 L2 中存活到被消费时，这一策略才有效。

调优重叠窗口

重叠比例由依赖启动信号发出与真正依赖就绪之间的时间间隔决定。更大的间隔意味着更大的潜在隐藏空间，但前提是资源保持不拥堵。

如果信号发得太早，两个 kernel 会争夺 SM 发射槽、寄存器文件容量、共享内存分配和内存端口。这可能把每个 kernel 都压慢到总时间不降反升。如果预取过于激进，L2 缓存行会在被使用之前就被搅动逐出，DRAM 流量因回填而飙升。带宽随后被花在把被逐出的数据搬回来，抹掉了预取重叠的收益。

实际调优要借助硬件计数器：在最晚的安全点启动，只预取稳定/被复用的张量，并把 L2 命中率与内存队列压力作为主要的守护指标。